

Setor de Educação Profissional e Tecnológica Tecnologia em Análise e Desenvolvimento de Sistemas

DS141 - Estruturas de Dados II

07 - Percursos e Propriedades de Árvores

Prof. Alex Kutzke

Junho de 2021

Slides adaptados do material produzido
pelo Prof. Rodrigo Minetto

Roteiro

Percursos

Altura da árvore

Altura, número de nós e eficiência

Exercícios propostos

Percurso

- Um **percurso** corresponde a uma **visita** sistemática a cada um dos nós da árvore;
- Muitas operações em árvores binárias envolvem o percurso de todas as subárvores, com a execução de alguma ação em cada nó.

Percurso

- Por exemplo: se cada nó da árvore possui um campo que armazena o salário;
- Para realizar um “reajuste salarial”, precisamos alterar o valor de todos os nós;
- Nesse caso “visitar um nó” seria atualizar o valor armazenado;
- Não podemos esquecer de nenhum nó, nem queremos visitar um nó mais do que uma vez;
- **Nesse caso**, a ordem da visita não é importante:
 - Mas em algumas outras aplicações, queremos visitar os nós em certa ordem desejada.

Visita

Visitar um nó significa trabalhar com a informação do nó (por exemplo: imprimir, modificar, etc). Contudo, durante um percurso podemos passar várias vezes por alguns dos nós sem visitá-los.

Formas de Percurso

É comum percorrer uma árvore em uma das seguintes ordens:

- Pré-ordem (**R**,**E**,**D**);
- In-ordem (**E**,**R**,**D**);
- Pós-ordem (**E**,**D**,**R**);

Pré-ordem

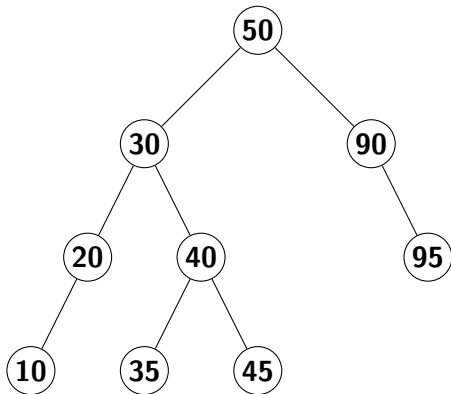
Pré-ordem (R, E, D):

1. Visitar a **raiz**;
2. Percorrer a **subárvore da esquerda** em pré-ordem;
3. Percorrer a **subárvore da direita** em pré-ordem.

Pré-ordem: exemplo

Percurso pré-ordem para a árvore abaixo:

50, 30, 20, 10, 40, 35, 45, 90, 95



- Segue os nós até chegar ao mais **"profundos"**, em **"ramos"** de subárvores da esquerda para a direita;
- Percurso em **profundidade**.

In-ordem

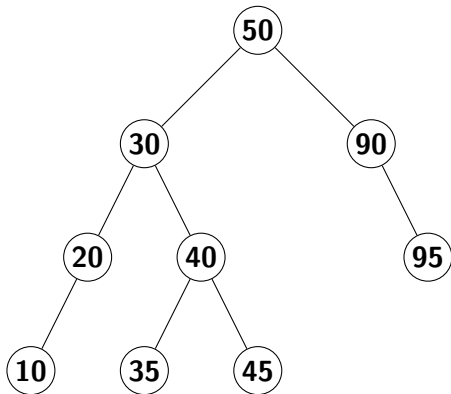
In-ordem (E, R, D):

1. Percorrer a **subárvore da esquerda** em in-ordem;
2. Visitar a **raiz**;
3. Percorrer a **subárvore da direita** em in-ordem.

In-ordem: exemplo

Percurso in-ordem para a árvore abaixo:

10, 20, 30, 35, 40, 45, 50, 90, 95



- Visita a raiz entre as ações de percorrer as duas subárvores;
- Percurso em **ordem simétrica**.

Pós-ordem

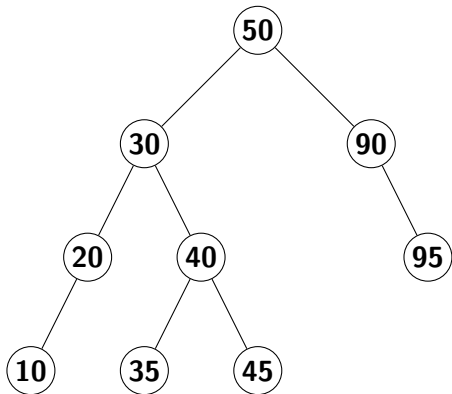
Pós-ordem (E, D, R):

1. Percorrer a **subárvore da esquerda** em pós-ordem;
2. Percorrer a **subárvore da direita** em pós-ordem.
3. Visitar a **raiz**;

Pós-ordem: exemplo

Percurso pré-ordem para a árvore abaixo:

10, 20, 35, 45, 40, 30, 95, 90, 50



- Visita a raiz **após** as ações de percorrer as duas subárvores;

Altura da árvore

- Existe apenas um caminho da raiz para qualquer nó:
 - Em outras palavras, o que isso quer dizer?

Altura da árvore

- Existe apenas um caminho da raiz para qualquer nó:
 - Em outras palavras, o que isso quer dizer?;
 - Não existem ligações entre nós “irmãos”. Quando isso ocorre, a estrutura deixa de ser uma árvore e se torna um *grafo*;

Altura da árvore

- Existe apenas um caminho da raiz para qualquer nó:
 - Em outras palavras, o que isso quer dizer?;
 - Não existem ligações entre nós “irmãos”. Quando isso ocorre, a estrutura deixa de ser uma árvore e se torna um *grafo*;
- Assim, **altura** é:
o comprimento do caminho mais longo, da raiz até uma das folhas.

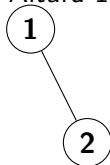
Altura da árvore

- Altura de uma árvore com um único nó é 0;
- Altura de uma árvore vazia é -1;

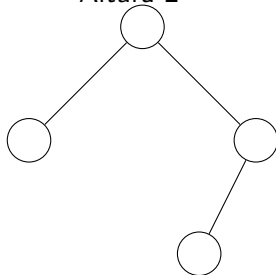
Altura 0



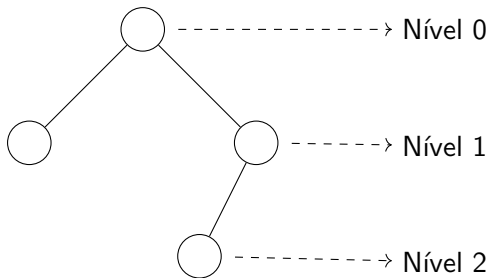
Altura 1



Altura 2

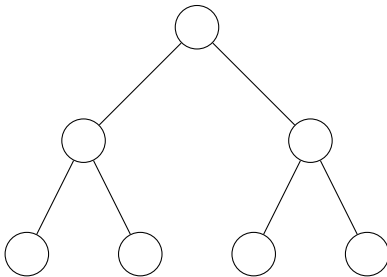


Níveis



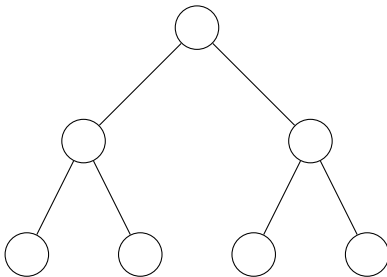
Áviores cheias: número de nós

Uma árvore *binária cheia (ou completa)* é aquela em que todos os nós internos têm duas subárvores e todos os nós folhas estão no último nível. uma árvore com um único nó é 0.



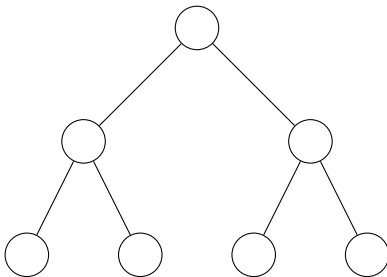
Árvore cheias: número de nós

Podemos notar que nesse tipo de árvore temos **um** nó no nível **0**, dois nós no nível **1**, **quatro** nós no nível **2**, **oito** nós no nível **3**, e assim por diante. Isto é, no nível n , temos 2^n nós.



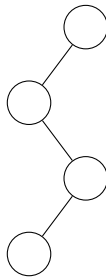
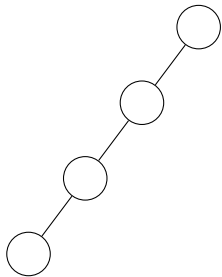
Árvore cheias: número de nós

É possível mostrar que uma árvore cheia de altura h tem um número de nós dado por $2^{h+1} - 1$. **Como?**



Árvores degeneradas

- Uma árvore é dita **degenerada** se todos os seus nós internos tem uma única subárvore associada;
- Apenas um nó por nível;
- Uma árvore degenerada de altura h possui $h + 1$ nós;



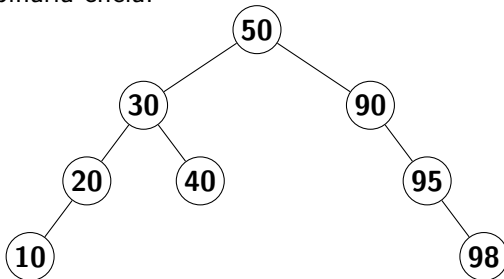
Eficiência X Altura

A altura de uma árvore é uma medida importante na avaliação da eficiência com que visitamos os nós da árvore.

Uma árvore binária com n nós tem uma altura mínima proporcional a $\log_2 n$ (caso da árvore cheia) e uma altura máxima proporcional a n (caso da árvore degenerada). A altura indica o esforço computacional necessário para alcançar qualquer nó da árvore.

Exercícios propostos

Exercício 1 Suponha a árvore binária abaixo. (a) ache o grau de cada um dos nós; (b) determine os nós folhas; e (c) complete a árvore com quantos nós forem necessários para transformá-la em uma árvore binária cheia.



Exercícios propostos

Exercício 2 Escreva funções que recebam uma árvore binária inteira como entrada e a imprima com os seguintes tipos de percurso:

- pré-ordem;
- in-ordem;
- pós-ordem.

Exercícios propostos

Exercício 3 Escreva uma função que conte o número de nós de uma árvore binária. Utilize o seguinte protótipo para a sua função:

```
int conta_nos(Arvore a);
```

Exercício 4 Escreva uma função que imprime e retorna o maior valor inteiro em uma árvore binária composta por valores inteiros. Utilize o seguinte protótipo para a sua função:

```
int max_arvore(Arvore a);
```

Exercícios propostos

Exercício 5 Escreva uma função que calcula a altura de uma árvore binária. Utilize o seguinte protótipo para a sua função:

```
int altura_arvores(Arvore *a);
```

Exercício 6 Escreva uma função que conta o número de nós folhas em uma árvore binária. Utilize o seguinte protótipo para a sua função:

```
int nos_folha_arvore(Arvore *a);
```

Referências utilizadas na apresentação I